UQAC
Université du Québec à Chicoutimi

日本語テキストの難易度予測

Prédiction de difficulté de textes japonais

Ismael El Amrani
Département d'Informatique et Mathématique de l'UQAC

1 — Le problème

Avez-vous déjà essayé d'apprendre le japonais, sans savoir si un texte est adapté à votre niveau ?

Le **JLPT** (Japanese Language Proficiency Test) définit 5 niveaux de maîtrise :

N5 débutant | N4 | N3 | N2 | N1 avancé

Un **texte japonais** combine kanji, hiragana, katakana et grammaire à des niveaux variés, rendant l'évaluation manuelle difficile.

Objectif : prédire automatiquement la difficulté en 3 classes :

easy | medium | hard

Texte japonais → Features → Classifieur → easy / medium / hard

2 — Données et ressources

Trois **ressources JLPT** constituent la base de connaissance :

- **Vocabulaire** — listes JLPT par niveau N5→N1
- **Kanji** — liste jōyō annotée par niveau JLPT
- **Grammaire** — patterns JLPT compilés manuellement

Corpus d'entraînement : **900 phrases** annotées manuellement (**300** par classe).

Colonnes : `text` `label` `target_jlpt`

Split **train/test** : **80/20** → **720** train / **180** test.

Validation — jeu séparé, jamais vu à l'entraînement :

99 phrases : 33 easy 33 medium 33 hard

3 — Régression Logistique

Le classifieur utilisé est une **Régression Logistique multiclasse** . En 3 classes (`easy` , `medium` , `hard`), on calcule d'abord un **score** par classe `k`

La validation finale s'effectue sur le jeu de validation séparé.

$$z_k = w_k^T x + b_k$$

Avec `k ∈ {easy, medium, hard}` , `x` = vecteur de features, (w_k, b_k) = paramètres appris pour la classe k.

Puis on obtient une distribution de probabilités par la **softmax** :

$$P(y=k \mid x) = e^{z_k} / \sum_j e^{z_j}$$

- Chaque `P(y=k|x)` est dans `[0, 1]` et représente la confiance du modèle pour la classe k.

La classe affichée (`Prediction: ...`) est celle de probabilité maximale (**argmax**) :

$$\hat{y} = \text{argmax}_k \; P(y=k \mid x)$$

En multi-classes, softmax normalise pour que la somme fasse 1:

$$P(easy) + P(medium) + P(hard) = 1$$

La classe prédite est celle de probabilité maximale (**argmax**). Un **score 0–100** est ensuite calculé à partir de P(easy) pour résumer la facilité du texte.

$$Score = round(P(easy) \times 100)$$

4 — Extraction de features linguistiques

Chaque texte est converti en un **vecteur numérique explicable** :

Ratios d'écriture
`ratio_kanji` `ratio_hira` `ratio_kata`

Structure
`punct_count` `sent_count` `comma_count`

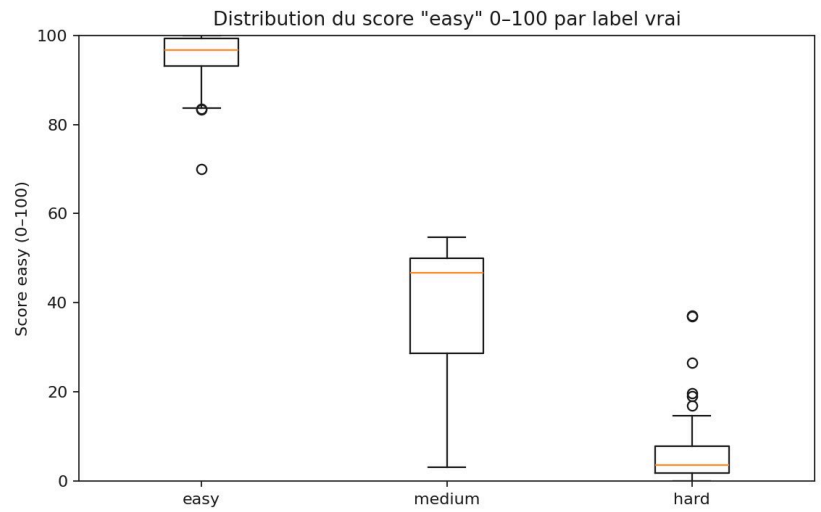
Signaux JLPT vocab
`vocab_ratio_N5` `vocab_ratio_N3` `vocab_ratio_N1`

Kanji + Grammaire
`kanji_max_level` `grammar_ratio_N2` `kanji_ratio_N1`

La **normalisation NFKC** est appliquée avant extraction. C'est une forme de normalisation Unicode qui réduit les variantes spécifique : `ｶﾀｶﾅ ⇔ カタカナ`

Score 0–100 : synthèse de la probabilité softmax de la classe easy .

$$Score = round(P(easy) \times 100)$$



Distribution du score 0–100 par label réel — séparation nette entre les classes

5 — Modèle et entraînement

Pipeline **scikit-learn** combinant deux familles de signaux :

① Features explicables
Ratios kana/kanji, signaux JLPT N5→N1, ponctuation…

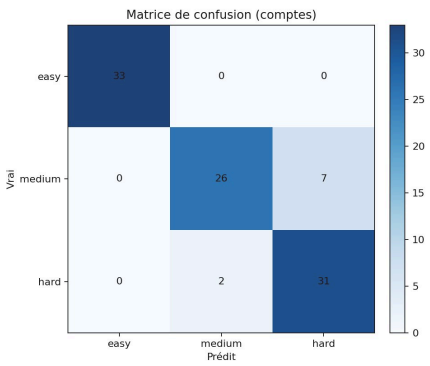
② Représentation texte
TF-IDF de n-grammes de caractères, sans tokenisation obligatoire

Features numériques ⊕ TF-IDF char n-grams → Régression Logistique → Prédiction + score

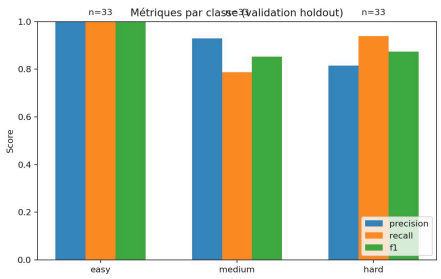
6 — Résultats

Évaluation sur le **jeu de validation** (99 phrases, jamais vues à l'entraînement) :

100%	85%	87%
F1 — easy	F1 — medium	F1 — hard



Matrice de confusion (validation holdout)



Précision, rappel et F1 par classe

La classe easy est parfaitement classée (33/33). La classe medium présente 7 confusions avec hard , ce qui est attendu : c'est une classe "entre-deux" difficile à délimiter.

7 — Limites et améliorations

Principale **limite** : la classe medium est la plus difficile à capturer :

- Elle se situe naturellement entre easy et hard
- Les features "proportions JLPT" peuvent manquer de signal sur la forme (style, tournures)
- Un corpus petit ou bruité amplifie l'instabilité de cette classe

Techniques d'amélioration du modèle :

- **Lemmatisation (SudachiPy)** : les listes JLPT sont en **forme dictionnaire** (ex. 食べる), alors que le texte contient des **formes conjuguées** (食べた, 食べます…). SudachiPy extrait la forme de base de chaque token pour mieux matcher le vocabulaire JLPT et améliorer la qualité des features. Sans SudachiPy, des heuristiques de déconjugaison prennent le relais (〜ます → forme en る, etc.).
- **Stress-test** : génère des phrases artificielles en répétant un terme JLPT ciblé (ex. `無料`) pour vérifier que le modèle ne sur-prédit pas la difficulté d'un seul mot. Les résultats sont exportés dans un CSV pour analyse.

8 — Continuité

Par la suite, il sera possible de :

- Affiner la **granularité** : passer de 3 classes à 5 (N5→N1) pour coller aux niveaux JLPT officiels
- Déployer une **interface web** permettant de coller un texte et d'obtenir son niveau estimé en temps réel
- Intégrer des modèles de langue pré-entraînés (**BERT japonais** — cl-tohoku/bert-base-japanese) pour capturer le contexte sémantique
- Enrichir **le corpus** avec des textes annotés publics (NHK, NHK Web Easy,etc..)

9 — Remerciements

- Merci à Sara Séguin d'avoir accepté de superviser ce projet.
- Merci aux auteurs des ressources ouvertes : Kaggle, GitHub (NHV33) et JLPT Sensei.

ieamrani@etu.uqac.ca

10 — Bibliographie

Pedregosa et al. (2011). Scikit-learn: Machine Learning in Python. *JMLR*, 12, 2825–2830.

Devlin et al. (2019). BERT: Pre-training of Deep Bidirectional Transformers. *NAACL-HLT 2019*.

Takaoka et al. (2018). Sudachi: a Japanese Tokenizer for Business. *LREC 2018*.

Pourtaud, R. (s.d.). *JLPT Words by Level* [Jeu de données Kaggle]. Saisi le 7 avril 2026 de https://www.kaggle.com/datasets/robinpourtaud/jlpt-words-by-level/data

Vincent, N. H. (NHV33). (s.d.). *Joyo kanji compilation* [Dépôt GitHub]. Saisi le 7 avril 2026 de https://github.com/NHV33/joyo-kanji-compilation/tree/master

Two Wheel Cruise. (s.d.). *JLPT Sensei — Member Benefits* [Ressource en ligne]. Saisi le 7 avril 2026 de https://jlptsensei.com/member-benefits/